

Arquitetura IIAE-BR — Documentação Técnica

IIAE-BR — Interoperabilidade de Agentes Inteligentes no E-commerce Brasileiro. Este documento descreve a arquitetura em **5 camadas** implementada no projeto, suas responsabilidades, componentes e o fluxo de mensagens entre agentes.

1. Visão Geral

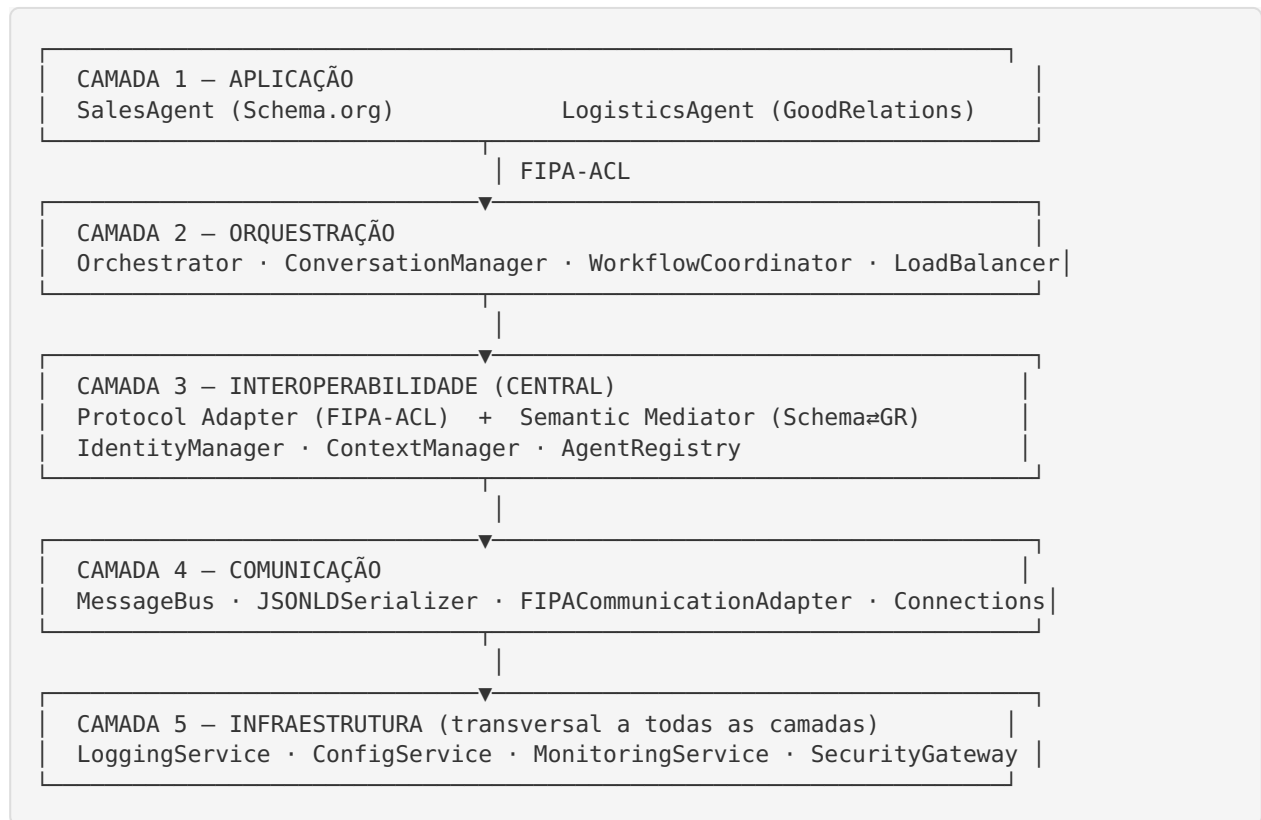
A arquitetura busca garantir a interoperabilidade entre agentes inteligentes que utilizam **vocabulários semânticos distintos** — agentes de venda falam **Schema.org** e agentes de logística falam **GoodRelations** — comunicando-se por meio do protocolo **FIPA-ACL** com serialização **JSON-LD**.

O princípio central é a **separação de responsabilidades em camadas**, onde a camada de **Interoperabilidade** (camada 3) atua como ponto único de adaptação de protocolo e mediação semântica, isolando os agentes de aplicação das diferenças de vocabulário.

Escopo e restrições de projeto

Dimensão	Escolha	Observação
Vocabulários	Schema.org e GoodRelations	nenhum outro é utilizado
Protocolo de agentes	FIPA-ACL	sem MCP, A2A ou ACP
Serialização	JSON-LD	sem XML ou FIPA-SL
Protocolos de interação	Request, Query, Contract-Net, Subscribe	máquinas de estado

2. Diagrama de Camadas



3. Responsabilidades por Camada

Camada 1 — Aplicação (src/layer_1_application/)

Agentes de domínio do marketplace.

Componente	Vocabulário	Responsabilidade
SalesAgent	Schema.org	Catálogo, ofertas e pedidos (Product , Offer , Order , OrderItem , PaymentChargeSpecification). Responde a query-if / query-ref , cfp e request .
LogisticsAgent	GoodRelations	Cálculo de frete e prazos (BusinessEntity , Product-OrService , UnitPriceSpecification , DeliveryMethod).

Cada agente conhece **apenas o seu vocabulário**; a tradução é delegada à camada 3.

Camada 2 — Orquestração (`src/layer_2_orchestration/`)

Componente	Responsabilidade
Orchestrator	Roteamento inteligente, fila de prioridade por performativa, integração com segurança e gestão de conversas.
ConversationManager	Ciclo de vida das conversas (aberta, aguardando, concluída, expirada), timeout e histórico.
WorkflowCoordinator	Fluxos multi-agente (ex.: <code>purchase</code> = consulta → frete → fechamento).
LoadBalancer	Distribuição entre instâncias do mesmo papel (round-robin / least-loaded).

Camada 3 — Interoperabilidade (`src/layer_3_interoperability/`) — CENTRAL

Protocol Adapter (`protocol_adapter/`)

Componente	Responsabilidade
FIPAACLAdapter	Ponto de entrada/saída. Integra parsing, validação, mapeamento de performativas, máquinas de estado e serialização.
MessageParser	Análise e validação estrutural das mensagens FIPA-ACL (objeto ou JSON-LD).
PerformativeMapper	Mapeia performativas → atos comunicativos e valida respostas coerentes.
InteractionProtocolEngine	Máquinas de estado dos protocolos: <code>fipa-request</code> , <code>fipa-query</code> , <code>fipa-contract-net</code> , <code>fipa-subscribe</code> .

Semantic Mediator (semantic_mediator/)

Componente	Responsabilidade
SemanticMediator	Orquestra a tradução, detecta o vocabulário de origem e reporta overhead.
SchemaOrgManager	Construção/validação de entidades Schema.org (rdflib).
GoodRelationsManager	Construção/validação de entidades GoodRelations (rdflib).
OntologyMapper	Tradução bidirecional Schema.org $\rightleftharpoons$ GoodRelations, com cache semântico configurável e medição de overhead de tradução .

Suporte

Componente	Responsabilidade
IdentityManager	Identities de agentes (AID), credenciais e vocabulário declarado.
ContextManager	Contexto conversacional (vocabulários dos participantes, necessidade de tradução).
AgentRegistry	Diretório de agentes (descoberta por papel, serviço ou vocabulário).

Camada 4 — Comunicação (src/layer_4_communication/)

Componente	Responsabilidade
CommunicationMessageBus	Barramento de mensagens (publicação/assinatura por tópico).
JSONLDSerializer	Serialização/deserialização JSON-LD das mensagens FIPA-ACL.
FIPACommunicationAdapter	Transporte: conexão, envio e recepção com round-trip de serialização.
ConnectionManager	Gestão de conexões com limite máximo.

Camada 5 — Infraestrutura (`src/layer_5_infrastructure/`)

Componente	Responsabilidade
LoggingService	Configuração e obtenção de loggers padronizados.
ConfigService	Carrega <code>config/iaae_br_config.yaml</code> , expõe acesso por camada e feature flags.
MonitoringService	Registra métricas e overhead por camada ; health checks.
SecurityGateway	Autorização básica e validação de mensagens.

4. Mapeamento Semântico Schema.org ⇌ GoodRelations

O `OntologyMapper` realiza a tradução bidirecional dos principais tipos e propriedades do e-commerce:

Schema.org	GoodRelations
Product	ProductOrService
Offer	Offering
Order	Order
OrderItem	OrderItem
PaymentChargeSpecification	UnitPriceSpecification
Person / Organization	BusinessEntity
sku / productID	hasStockKeepingUnit
price	hasCurrencyValue
priceCurrency	hasCurrency
availability	hasInventoryLevel

Cache semântico e overhead de tradução

- O cache pode operar em **modo realista** (acerto se a chave já existe) ou em **modo experimento** (`forced_hit_rate`), que reproduz proporções exatas de acerto — usado nos cenários de cache **0%, 50%, 60%, 80% e 95%**.

- Cada tradução tem seu **tempo medido**: um cache hit tem custo desprezível (~0,03 ms), enquanto um cache miss incorre no custo da tradução efetiva (~1,8 ms). O overhead acumulado é exposto para análise.

5. Fluxo de Mensagem (exemplo: cotação de frete)

1. SalesAgent (Schema.org) cria um Product e solicita frete.
 ACLMessage(REQUEST, protocol=fipa-request)

 2. CAMADA 4: serializa a mensagem em JSON-LD e transporta pelo barramento.

 3. CAMADA 3 / Protocol Adapter:
 -  MessageParser valida a estrutura;
 -  PerformativeMapper classifica o ato (DIRECTIVE);
 -  InteractionProtocolEngine: INITIATED  PENDING.
 - 
 4. CAMADA 3 / Semantic Mediator:
 -  detecta que o destino (logística) fala GoodRelations;
 -  traduz Product (Schema.org)  ProductOrService (GoodRelations);
 -  mede o overhead de tradução (depende do cache).
 - 
 5. CAMADA 2 / Orchestrator:
 -  resolve o papel "logistics" no AgentRegistry;
 -  LoadBalancer (round-robin) escolhe a instância;
 -  registra a conversa no ConversationManager.
 - 
 6. CAMADA 1: LogisticsAgent calcula o frete e responde com UnitPriceSpecification (GoodRelations) via INFORM.

 7. O caminho inverso traduz UnitPriceSpecification  PaymentChargeSpecification (Schema.org) antes de entregar a resposta ao SalesAgent.
- Em todas as etapas, a CAMADA 5 registra logs, métricas e overhead por camada.

6. Medição de Overhead por Camada

A integração com a simulação Monte Carlo adiciona, a cada transação, o custo de processamento de cada camada (src/simulation/layer_overhead.py):

Camada	Custo-base (ms)	Observação
1 — Aplicação	0,0	lógica de negócio já modelada nos agentes
2 — Orquestração	~0,25	roteamento + balanceamento + conversa
3 — Interoperabilidade	~0,40 + tradução	varia com o cache semântico
4 — Comunicação	~0,30	serialização JSON-LD + transporte
5 — Infraestrutura	~0,05	logging + monitoramento + segurança

O overhead da camada 3 cai à medida que a taxa de acerto do cache aumenta (ex.: ~2,2 ms a 0% → ~0,5 ms a 95%), evidenciando o impacto da mediação semântica no desempenho. Os resultados são exportados em CSV/JSON, no relatório textual (seção 6) e no gráfico `overhead_por_camada.png`.

7. Configuração (`config/iaae_br_config.yaml`)

O arquivo declara as 5 camadas e os parâmetros de cada componente (ontologias dos agentes, tamanho de fila/timeout da orquestração, protocolos e serialização da interoperabilidade, cache do mediador semântico, barramento de comunicação e serviços de infraestrutura). É carregado pelo `ConfigService`.

8. Testes

Os testes em `tests/test_layers.py` cobrem: round-trip JSON-LD, máquinas de estado dos protocolos FIPA-ACL, tradução semântica bidirecional, cache configurável, roteamento/prioridade da orquestração, agentes de aplicação, serviços de infraestrutura e a medição de overhead por camada na simulação.

```
python -m pytest tests/ -v
```